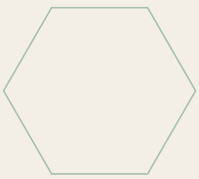
The Fulfillment *Playbook.*

From card-charged to activated customer in under ten minutes. The seller stack. The activation-key system. The email templates. The refund flow. The ongoing cadence.



PART ONE

The Fulfillment Flow

GEM is a digital product sold one-time, fulfilled instantly, supported lightly. Every order should move from *card charged* to *customer using the product* in under ten minutes with zero manual work from you. This part is the high-level map before the rest of the playbook drills into each step.

The happy path — what happens for every sale

→ **1. Buyer pays on Gumroad.** Buyer hits "Get GEM — $9" on the landing page, types card details, clicks pay. Gumroad charges, generates the unique license key (auto), and triggers download.

→ **2. Buyer downloads the ZIP.** Gumroad serves the bundled ZIP directly from its CDN. No action from you. The ZIP contains gem.html, the user manual PDF, the LICENSE.txt with their key, and a WELCOME.md.

→ **3. Gumroad emails the receipt + key.** The receipt email includes the license key inline (handy for buyers who lose the ZIP). The same email contains a re-download link valid forever.

→ **4. Buyer opens gem.html.** The product runs locally in their browser. No account, no server-side activation — the license is on the seller side, not the product side (see Part 4).

→ **5. You log the sale & bump the counter.** Update SEATS_REMAINING in landing.html as the Founder cohort fills. Add the order to your seller ledger (Part 8). Send the 24-hour follow-up email if they haven't opened gem.html yet (optional).

What every buyer receives (the four artifacts)

Artifact	Format	Purpose
gem.html	Single 330 KB HTML file	The product. Runs locally in any modern browser.
GEM-User-Manual.pdf	11-page PDF	Setup, daily workflow, troubleshooting, license terms.
LICENSE-{order-id}.txt	1 KB plain text	Their unique license key, purchase date, tier, support eligibility.
WELCOME.md	~2 KB markdown	60-second getting-started: open gem.html, configure Ollama or cloud key, run first pipeline.

Service-level promises. Auto-fulfillment within seconds of payment. License key delivered in the same email. First support reply within 48 hours business days. Founder's Edition buyers (first 100) get a direct line to the founder.

PART TWO

The Seller Stack

You don’t need a custom backend. Pick one merchant-of-record platform and let it do the heavy lifting: checkout, tax, fraud, license keys, refunds, file hosting, email receipts. Below is the decision matrix and the recommended setup.

Platform	Fee	License keys	Tax / VAT	Verdict for $9 indie launch
Gumroad	~10% per sale	Built-in toggle	Handled by Gumroad	★ Recommended. Cheapest setup, fastest go-live.
Lemon Squeezy	5% + $0.50	Built-in API	Handled (MoR)	Best if you outgrow Gumroad. Better UX, higher fee floor.
Stripe + custom	~2.9% + $0.30	You build	You handle	Highest control, highest build cost. Skip for the first 500 sales.
AppSumo	~70% revenue share	Built-in	Handled	Use only if you want a 1,000-sale spike. Painful margin.

Gumroad setup — eight steps

- **Step 1.** Create a Gumroad account at `gumroad.com`. Free tier is fine until you cross $5K total revenue.
- **Step 2.** Create the product: **Type:** Digital product. **Title:** "GEM Content Engine — Founder’s Edition". **Price:** $9.00.
- **Step 3.** Upload the bundled ZIP (see Part 3 for what goes in it). Gumroad hosts the file on its CDN. Maximum file size: 16 GB free tier (you’ll use ~400 KB).
- **Step 4.** Enable license keys: Product settings → "Generate license keys for this product" → toggle on. Gumroad now auto-creates a unique key per sale, sends it with the download.
- **Step 5.** Customize the receipt email: Settings → Emails → Receipt. Paste the template from **Appendix A**. Include the license key with `{{license_key}}`.
- **Step 6.** Set the product URL on the live site. The Get GEM CTA in `landing.html` already points to `https://hussain412.gumroad.com/l/qlqsa` — update the slug to match yours.
- **Step 7.** Connect your payout method: Gumroad → Settings → Payouts. Choose direct deposit, PayPal, or Stripe Express. First payout lands on a Friday 7+ days after first sale.
- **Step 8.** Test the flow yourself: place a $9 self-purchase, confirm the ZIP arrives with a valid key, then refund yourself. End-to-end smoke test takes 10 minutes.

Lemon Squeezy alternative (when you outgrow Gumroad)

Roughly the same flow with three differences worth knowing: (1) Lemon Squeezy is a true merchant of record in more jurisdictions, so VAT/GST is fully off your plate; (2) license-key validation has a proper REST API at `api.lemonsqueezy.com/v1/licenses/validate`; (3) the UX is better-looking out of the box. Trade-off is the 5% + $0.50 floor — on a $9 sale you net $7.95 vs $8.10 on Gumroad.

Stripe + custom flow (when you have the engineering bandwidth)

Use this only after 500+ sales and a real need for control (custom upsells, white-label, B2B invoicing). Minimum build: a small serverless function that listens to `checkout.session.completed`, generates an HMAC license key (Part 4), stores the order, and triggers an email with the ZIP link via Postmark or Resend. About a weekend of work if you've done it before.

PART THREE

The Buyer Package

Everything the buyer gets ships as a single ZIP. Small, clean, no fluff. Below is the exact structure to upload to Gumroad.

```
GEM-Content-Engine-v1.7.zip   (~370 KB total)
■■■ gem.html                        ← the product (single-file app)
■■■ GEM-User-Manual.pdf             ← 11-page setup + workflow guide
■■■ WELCOME.md                      ← 60-second start guide (see below)
■■■ LICENSE.txt                     ← buyer's license key + terms summary
■■■ CHANGELOG.txt                   ← what's in v1.7 + roadmap teaser
```

File-by-file rationale

- **gem.html.** The single 330 KB HTML file. Self-contained, runs anywhere. **Do not minify** — buyers need to read it for trust.
- **GEM-User-Manual.pdf.** The 11-page user manual regenerated for each release (Part 5 of Sales Strategy explains the doc pipeline).
- **WELCOME.md.** A 2 KB markdown file with three things: (1) "Open gem.html in your browser." (2) "Paste your Ollama URL or cloud key in Settings." (3) "Hit Generate Content Drop." That's it. Buyers want the on-ramp to be a paragraph, not a chapter.
- **LICENSE.txt.** Plain-text file with the buyer's license key, their tier (Founder's / Builder / Operator / Studio), the purchase date, and a five-line summary of license terms (linking to `terms.html` on the live site).
- **CHANGELOG.txt.** A trimmed customer-facing version of `CHANGELOG-v1.7.md` — "what's new and what's coming." Sets expectations that updates land in their inbox.

Naming convention

Always include the version in the ZIP filename: **GEM-Content-Engine-v1.7.zip**. When you ship v1.8, re-upload as **GEM-Content-Engine-v1.8.zip** — Gumroad serves the latest file to *all past buyers* automatically because lifetime updates are part of the license. Past versions stay in your local archive but should not stay on Gumroad.

Optional: split into core + extras

If you ever cross the 5 MB mark (custom templates, large branding kit, etc.), split into two Gumroad downloads: **GEM-Core-v1.7.zip** — product + manual + WELCOME + LICENSE (~400 KB). **GEM-Extras-v1.7.zip** — branding kit, niche templates, design files (anything optional). Both are gated behind the same purchase.

PART FOUR

Activation Keys

GEM's product philosophy is *privacy-first, no server, no account* — so the activation key is intentionally a soft one. It exists for the seller's records (audit trail, support eligibility, abuse detection), not as a runtime lock on the product. Buyers can run gem.html without entering anything. They *reference* their license key when contacting support or upgrading tiers.

Why have keys at all? Three reasons. (1) Audit — you know who bought what when. (2) Support eligibility — the buyer quotes their key, you verify it in your seller ledger. (3) Abuse signal — if the same key appears in two support tickets from different emails, you have a clear resale signal.

Three approaches — pick one

A. Gumroad built-in (recommended for first 1,000 sales). Toggle the *Generate license keys for this product* setting. Gumroad auto-creates a key per sale, includes it in the receipt email, and exposes a verify endpoint at `api.gumroad.com/v2/licenses/verify`. Zero engineering.

B. Custom HMAC (recommended if you ever leave Gumroad). Generate a deterministic key from `HMAC-SHA256(secret, order_id)`. You can re-derive any key from the order ID and your secret, so you don't need a database. See the code snippet below.

C. Manual UUIDs (only for < 20 sales). Run `python -c "import uuid; print(uuid.uuid4())"` per sale and paste the result into a spreadsheet. Fine for a private beta, not for scale.

Recommended key format

```
GEM-XXXX-XXXX-XXXX-XXXX     (e.g. GEM-7B3C-91F0-4A2E-D8B6)
```

Four hex groups of 4 characters, prefixed with **GEM-**. Easy to read aloud over a call. Sixteen hex chars = 64 bits of entropy — way more than you need for $9 indie product, but no reason not to.

Python — custom HMAC key generator

```
# scripts/keygen.py
import hmac, hashlib, os

SECRET = os.environ['GEM_KEYGEN_SECRET'].encode()   # keep this off Git

def make_key(order_id: str) -> str:
    raw = hmac.new(SECRET, order_id.encode(), hashlib.sha256).hexdigest()
    chunks = [raw[0:4], raw[4:8], raw[8:12], raw[12:16]]
    return 'GEM-' + '-'.join(c.upper() for c in chunks)

def verify_key(order_id: str, claimed_key: str) -> bool:
    return hmac.compare_digest(make_key(order_id), claimed_key)

# Usage at sale time:
#   key = make_key('gumroad-order-12345')
#   write key into LICENSE.txt, email to buyer.
# Usage at support time:
```

```
#   if verify_key(order_id, buyer_quoted_key): grant support.
```

Seller ledger — the spreadsheet you actually keep

Column	Why it matters
order_id	The Gumroad / Stripe order reference. Primary key.
license_key	The key you emailed the buyer. Deterministic if HMAC.
email	Customer email. Used for support and revocation.
tier	Founder's / Builder / Operator / Studio. Drives support depth.
purchased_at	ISO date. Used for the 14-day refund window.
version_at_purchase	e.g. v1.7. So you know which manual they got.
status	active / refunded / revoked. Filter on this in every support ticket.
notes	Free-text. Refund reason, upgrade history, escalation flags.

A Google Sheet works for the first 500 sales. After that, move to a SQLite file or Airtable — Gumroad's export covers most of this but doesn't carry your *notes* column.

Optional — gentle in-product license display

If you want gem.html to *display* the license key (not gate behind it), paste this snippet inside the existing Settings panel. Pure display, no validation, no server call — true to the privacy promise.

```
<!-- Drop inside the Settings tab in gem.html -->
<div class="gem-license">
  <label for="gem-license-key">License key</label>
  <input id="gem-license-key" type="text"
         placeholder="GEM-XXXX-XXXX-XXXX-XXXX"
         pattern="GEM-[A-F0-9]{4}-[A-F0-9]{4}-[A-F0-9]{4}-[A-F0-9]{4}">
  <small>From your LICENSE.txt. Stored locally only. Not validated online.</small>
</div>
<script>
  const el = document.getElementById('gem-license-key');
  el.value = localStorage.getItem('gem_license') || '';
  el.addEventListener('change', () =>
    localStorage.setItem('gem_license', el.value.trim()));
</script>
```

PART FIVE

Delivery

On Gumroad, delivery is automatic. The buyer downloads the ZIP from a unique URL, and a receipt email lands in their inbox containing the license key plus a permanent re-download link. Your job is to make the receipt email and the WELCOME.md feel like a person wrote them, not a vending machine.

The three emails the buyer should receive

When	Email	Goal
T+0 seconds	Receipt + license key	Confirm purchase, surface key, reduce re-download friction.
T+30 minutes (if first run not detected)	“First-run nudge” (optional)	Catch buyers who downloaded and got distracted before opening gem.html.
T+24 hours	“How’s it going?” follow-up	Open the support conversation. Most buyers will reply with one specific question.
T+14 days	“Welcome past the refund window”	Celebrate the milestone, ask for a one-line testimonial. Optional but high-ROI.

Full email templates are in Appendix A. Use Gumroad’s built-in receipt customizer for the T+0 email; everything past that goes through whatever email tool you already use (ConvertKit, Buttondown, Mailerlite, Resend).

Re-delivery (when someone loses the file)

Gumroad already handles this — every receipt email contains a permanent re-download link. If a buyer emails anyway: ask for the email they purchased with, look up the order in Gumroad, click *Resend receipt*. Total time: 90 seconds. Don’t treat re-delivery as a chore — it’s a chance to remind them v1.7 is the latest and they’re still on lifetime updates.

PART SIX

Post-Purchase Support

Support for a $9 product needs to be cheap enough to fit the price. Keep response targets generous and channel the bulk of questions to self-serve.

Channel	When to use	Response target
Email (hello@thegeminfo.com)	Setup issues, refund requests, account questions	Within 48 business hours
Discord community	Tips, sharing outputs, feature requests, peer help	Best-effort; community helps community
Founder DM (Founder's Edition only)	Strategic feedback, roadmap input, escalations	Within 7 days
Knowledge base / FAQ	Most questions are answered here first	Live on the site

Common support questions & canned replies

Q. "Ollama says connection failed."

A. They forgot CORS. Reply with one line: "Quit Ollama and relaunch with `OLLAMA_ORIGINS="*" ollama serve`. Then refresh gem.html and click Test Connection again."

Q. "Trending Mode only shows Reddit posts."

A. Expected behavior. Google Trends is CORS-blocked; we fall back to r/popular. Documented in the live FAQ and Chapter 7 of the manual. If they want unrestricted Google Trends, point them to the one-line proxy in the manual.

Q. "My API key is showing 'Invalid.'"

A. Ask which provider. Confirm the prefix: `sk-` (OpenAI), `sk-ant-` (Anthropic), `sk-or-` (OpenRouter). If correct, ask them to paste it freshly — pasting from password managers sometimes inserts trailing whitespace.

Q. "Can I use GEM for client work?"

A. Yes — Founder's Edition includes a commercial-use license. Point them to `terms.html § 5` on the live site for the formal grant.

Q. "My PDF export is blank."

A. Browser print blocked custom fonts. Switch to Chrome / Edge and re-export. If still blank: Settings → Export → "Use system fonts" toggle.

Q. "Will there be a v2.0?"

A. Yes. Daily auto-brief, approvals, direct publish, performance loop are all on the roadmap and Founder's Edition buyers will receive a substantial discount on v2.0 if it ships as a separate paid upgrade. Linked from the live `#roadmap` section.

PART SEVEN

Refunds, Revocations, and Abuse

The shorter your refund window and the cleaner your policy, the fewer disputes you'll see. The Terms of Service (live on the site) already locks in the policy below — this part is the operational side: what you actually click.

Refund policy (already published on terms.html § 7)

- **14-day no-questions-asked refund** on Founder's and Builder tiers.
- **30-day refund** on Operator tier, conditional on the buyer documenting the issue.
- **Studio tier refunds are pro-rated** against any setup-call or onboarding hours already delivered.
- **Refunds are processed via the same merchant of record** the purchase used (Gumroad / Lemon Squeezy).
- Refund triggers automatic license revocation in the seller ledger (Part 4).

Refund procedure — five clicks on Gumroad

- **Step 1.** Open Gumroad → Sales → find the order by email or order ID.
- **Step 2.** Click *Refund*. Gumroad reverses the charge and emails the buyer.
- **Step 3.** In your seller ledger, set `status = refunded`, paste a one-line note (e.g. "didn't work for their niche").
- **Step 4.** Mark the license key as revoked (Part 4) so any future support ticket quoting that key gets the gentle reply: "I can see this order was refunded. Happy to help still, but the license key isn't active."
- **Step 5.** Update the public seat counter on landing.html if the refunded sale was inside the Founder cohort (e.g. seats remaining goes from 73 back to 74).

Abuse detection — patterns to watch

- **Same key, two emails.** Resale. Revoke the key and email both addresses.
- **Refund-then-redownload.** Gumroad already blocks downloads post-refund, but if you used a custom delivery, build the same gate in.
- **Bulk purchases from disposable email addresses.** Sometimes legitimate (gift cards, agency buying for seats), sometimes credit-card fraud. Gumroad flags most; review unusual orders manually.
- **Chargebacks.** Always reply to the dispute with the receipt email + license key + download log. Most disputes resolve in your favor with a single screenshot of the buyer downloading the file.

Tier upgrade path (Builder → Operator → Studio)

If a Founder's buyer asks to upgrade later: don't build a separate product. Send them a one-time Gumroad discount link for **(new tier price** – **amount already paid)**. Update their tier in the ledger. They keep their

original license key — it now maps to the higher tier. Total operator time: ~5 minutes per upgrade.

PART EIGHT

Operational Cadence

Most of the work in selling GEM is the ongoing, low-energy rhythm — not the launch sprint. Below is the realistic cadence for a one-person operation past the launch window.

Cadence	Task	Time
Daily	Skim Gumroad sales notifications. Triage support inbox.	5–10 min
Daily	Reply to anything blocking a buyer (CORS errors, key questions).	10–20 min
Weekly	Update `SEATS_REMAINING` in `landing.html` as Founder cohort fills.	2 min
Weekly	Sync Gumroad sales into the seller ledger (CSV export).	10 min
Weekly	Ship one piece of organic content (Twitter thread, YouTube short, or blog).	60–90 min
Monthly	Refund the chargebacks worth refunding; dispute the rest.	15 min
Monthly	Tag a release in this repo if you shipped fixes — update `CHANGELOG-v1.7.md` or open `CHANGELOG-v1.8.md`.	20 min
Monthly	Re-run `scripts/regenerate_pdfs.py` if any PDF content changed; bundle new ZIP, re-upload to Gumroad.	10 min
Quarterly	Reconcile payouts. Lemon Squeezy / Gumroad handle the VAT side; you reconcile income tax in your jurisdiction.	30–60 min
Quarterly	Audit the seller ledger for revoked keys still active, stale notes, unsubscribes.	15 min
As needed	Crossed a price-ratchet threshold? Bump price in Gumroad, update landing.html copy, send announce email.	20 min

APPENDIX A

Email Templates

Paste these into Gumroad’s receipt customizer (template 1) and your email tool (templates 2–4). Replace bracketed placeholders. Keep the tone direct and operator-grade — these are not corporate auto-emails.

Template 1 — Order receipt + license key (T+0)

```
Subject: Your GEM Content Engine download + license key

Hi {{purchaser_name}},

Thanks for buying GEM Content Engine — Founder's Edition.
Your license key is below; tuck it somewhere safe.

    License key: {{license_key}}
    Tier:        Founder's Edition (first 100 buyers)
    Purchased:   {{purchased_at}}

Download (lifetime re-download from this link):
{{download_url}}

Two minutes to your first content drop:
  1. Unzip and open gem.html in your browser.
  2. Settings -> paste your Ollama URL or cloud API key.
  3. Hit Generate Content Drop.

Direct line for Founder's Edition buyers: hello@thegeminfo.com.
Reply to this email anytime.

-- The GEM team
thegeminfo.com
```

Template 2 — 24-hour follow-up

```
Subject: Did GEM open OK?

Hey {{first_name}},

Yesterday you grabbed GEM Content Engine. Three quick things:

1. The fastest first run is on Ollama (free, local). If you don't have it,
   the manual's Chapter 3 walks through the 5-minute install.

2. If you'd rather use a cloud LLM, paste any OpenAI / Anthropic /
   OpenRouter key in Settings. A full pipeline run costs about 2-5 cents.

3. If something's broken, hit reply. I read every email.

What I'd love back: one sentence on the first piece of content you ship
with GEM. I'm building the v1.8 roadmap from real feedback.

-- {{your_first_name}}
GEM Content Engine
```

Template 3 — Past-refund-window check-in (T+14 days)

```
Subject: 14 days in -- how's GEM treating you?

Hey {{first_name}},

Two weeks ago you bought GEM Content Engine.
You're officially past the no-questions refund window -- which means
you've already gotten enough value to keep it (or you forgot we exist;
either is fine).

If GEM has earned its keep on your channels, would you reply with a
single sentence I can quote on the site? Founder-cohort testimonials
are the most valuable thing I have right now.

If GEM hasn't earned its keep, tell me why. I'd rather refund you on
day 15 than keep $9 you regret.

-- {{your_first_name}}
```

Template 4 — Refund acknowledgment

```
Subject: Refund processed -- GEM Content Engine

Hi {{first_name}},

Refund processed via {{merchant_of_record}}; you should see the credit
back on your card within 5-10 business days.

Your license key ({{license_key}}) is now marked inactive in our records.
Please delete the gem.html file and the user manual from your machine.

Two quick asks (totally optional):
  - One sentence on what didn't work for you. Helps the next 100 buyers.
  - Permission to email you if v2.0 ships something you'd want.

Thanks for trying it.

-- {{your_first_name}}
GEM Content Engine
```

APPENDIX B

Code & Scripts

Three small scripts that automate the boring parts of fulfillment. Drop into `scripts/` alongside `regenerate_pdfs.py`.

B.1 — Package the buyer ZIP (Bash)

```
# scripts/package_buyer_zip.sh
#!/usr/bin/env bash
set -euo pipefail
VERSION="v1.7"
OUTPUT="GEM-Content-Engine-${VERSION}.zip"
STAGING="$(mktemp -d)"

cp gem.html "$STAGING/"
cp GEM-User-Manual.pdf "$STAGING/"
cp dist/WELCOME.md "$STAGING/"
cp dist/LICENSE.template.txt "$STAGING/LICENSE.txt"
cp dist/CHANGELOG.txt "$STAGING/"

(cd "$STAGING" && zip -r "$OLDPWD/$OUTPUT" .)
rm -rf "$STAGING"
echo "Built $OUTPUT ($(du -h "$OUTPUT" | cut -f1))"
```

B.2 — Render LICENSE.txt per order (Python)

```
# scripts/render_license.py
import sys, datetime, hmac, hashlib, os

SECRET = os.environ['GEM_KEYGEN_SECRET'].encode()

def make_key(order_id):
    raw = hmac.new(SECRET, order_id.encode(), hashlib.sha256).hexdigest()
    return 'GEM-' + '-'.join(raw[i:i+4].upper() for i in (0,4,8,12))

def render(order_id, email, tier='Founder's Edition'):
    today = datetime.date.today().isoformat()
    return f'''GEM Content Engine -- License

License key:    {make_key(order_id)}
Order ID:       {order_id}
Licensee:       {email}
Tier:           {tier}
Purchased:      {today}
Version:        v1.7

Lifetime license for personal and commercial content creation.
Lifetime updates within the v1.x major version.
Full terms: https://thegeminfo.com/terms.html
'''

if __name__ == '__main__':
    order_id, email = sys.argv[1], sys.argv[2]
```

```
    print(render(order_id, email))
```

B.3 — Verify a quoted key (Python one-liner)

```
$ python -c "\
from scripts.render_license import make_key; \
import os, sys; \
print(make_key(sys.argv[1]) == sys.argv[2])" \
  gumroad-order-12345  GEM-7B3C-91F0-4A2E-D8B6
```

Final note

The playbook above is the steady-state operating manual. The first 100 sales will be messier than this — and that's fine. Founder's Edition is for the cohort, not the spreadsheet. Once you cross 100 buyers, the playbook stops being theoretical and starts running itself.